

E-Learning JCB — Guion de Exposición Teórica (15 minutos)

~130 palabras/minuto • ~1.950 palabras • Solo exposición oral
La demostración práctica (15 min) se realiza a continuación en vivo

ESTRUCTURA

Bloque	Tema	PDF	Tiempo
1	Introducción y contexto	Pág. 1–2	1 min
2	Arquitectura general del sistema	Pág. 3	1.5 min
3	La base de datos: MongoDB y modelos	Pág. 4	1.5 min
4	Los estados de Reflex: el ciclo reactivo	Pág. 5	2 min
5	Páginas, componentes y patrones de UI	Pág. 6	1.5 min
6	Seguridad: autenticación y roles	Pág. 7	2 min
7	Mercado y oportunidad	Pág. 8	1 min
8	Modelo de negocio	Pág. 9	1 min
9	Viabilidad: técnica, económica y legal	Pág. 10	1.5 min
10	Ayudas al lanzamiento	Pág. 11	0.5 min
11	Conclusiones y próximos pasos	Pág. 12	1 min

BLOQUE 1 — INTRODUCCIÓN Y CONTEXTO

 1 minuto • ~130 palabras • Pág. 1-2 del PDF

Buenos días a todos.

Voy a presentaros **E-Learning JCB**, una plataforma de formación online construida íntegramente en Python. No hay ni una sola línea de JavaScript escrita manualmente.

Esto es posible gracias a **Reflex**, un framework que compila código Python en React + FastAPI automáticamente. Escribes Python puro; Reflex genera el frontend y el backend por debajo.

La plataforma **ya está desplegada en producción** en Reflex Cloud y accesible públicamente.

(Pág. 2 del PDF — métricas del proyecto)

Para tener la escala en mente:

- **39 archivos Python, 0 líneas de JavaScript manual**
- **30 páginas web, 33 rutas registradas**
- **10 clases de estado** gestionando toda la lógica
- **~18.000 líneas de código**

BLOQUE 2 — ARQUITECTURA GENERAL DEL SISTEMA

🕒 1.5 minutos • ~195 palabras • Pág. 3 del PDF • Módulo: DAW

(Pág. 3 del PDF — diagrama de arquitectura)

La arquitectura sigue un patrón en capas adaptado al paradigma reactivo de Reflex. Hay **cinco capas**:

1. **Navegador**: ejecuta React generado automáticamente. HTML/CSS/JS que yo nunca escribí.
2. **Páginas y componentes**: 30 archivos Python definiendo la UI.
3. **Estados**: 10 clases Python que almacenan datos y sincronizan la UI vía WebSocket.
4. **Servicios**: 4 módulos Python que encapsulan todas las operaciones con MongoDB.
5. **MongoDB Atlas**: base de datos NoSQL en la nube, 4 colecciones.

Las rutas dinámicas como `/courses/[course_id]` extraen el parámetro de la URL e inyectan su valor en el estado automáticamente.

Stack completo: **Python 3.14 + Reflex + MongoDB Atlas + Motor async + bcrypt + Granian + Redis.**

BLOQUE 3 — LA BASE DE DATOS: MONGODB Y LOS MODELOS

🕒 1.5 minutos • ~195 palabras • Pág. 4 del PDF • Módulo: DAW

(Pág. 4 del PDF — documento JSON real vs modelo Python)

¿Por qué MongoDB? Porque los cursos tienen estructura variable. Con SQL necesitaría nuevas tablas y JOINS costosos. Con MongoDB, un curso es un **documento JSON flexible** como el que veis en pantalla.

Mirad el documento de la izquierda: el instructor está completo —nombre y bio— dentro del mismo documento. Las lecciones también. Las reseñas también. Sin embargo, los estudiantes son solo tres IDs truncados: `"65a...e1d"` , `"65b...f2e"` .

Esto no es casual. El modelo Python de la derecha lo refleja exactamente: `instructor` , `lessons` y `reviews` están anotados como `# embebido` . Los `students` son `list[str]` con `# solo IDs` .

La regla que rige todo el esquema está al pie del slide: **si siempre se lee junto, se embebe. Si se consulta por separado, se referencia**. Las lecciones siempre se cargan con el curso: embebidas. Los estudiantes se consultan en sus propios perfiles: solo el ID.

Un detalle crítico que me costó un bug en producción: MongoDB almacena esos IDs como tipo `ObjectId` , no como `string`. Sin convertir con `str()` , la comparación `"65a...e1d" == ObjectId("65a...e1d")` devuelve `False` silenciosamente.

Los servicios usan `async / await` con Motor: mientras MongoDB responde, el servidor atiende otras peticiones sin bloqueos.

BLOQUE 4 — LOS ESTADOS DE REFLEX: EL CICLO REACTIVO

🕒 2 minutos • ~260 palabras • Pág. 5 del PDF • Módulo: DAW

(Pág. 5 del PDF — ciclo reactivo completo)

El **State** es el concepto central de Reflex. Mirad el diagrama superior: el usuario escribe en un campo → Reflex llama a `State.set_search_query()` → `@rx.var` recalcula → React re-renderiza. Cuatro pasos, cero código de infraestructura escrito por mí.

En el recuadro de la izquierda está el estado Python: `CourseState` tiene dos campos: `search_query: str` y `courses: list[dict]`. El método `filtered_courses` lleva el decorador `@rx.var` —lo que lo convierte en una variable computada—. El cálculo es un list comprehension: recorre todos los cursos y filtra los que contienen la query en el título. Se recalcula **solo cuando cambia** `search_query`.

```
class CourseState(rx.State):
    courses: list[dict] = []
    search_query: str = ""

    @rx.var
    def filtered_courses(self) -> list[dict]:
        q = self.search_query.lower()
        return [c for c in self.courses if q in c["title"].lower()]
```

En el recuadro de la derecha está la UI, también Python puro: un `rx.input` con `on_change=CourseState.set_search_query` —cuando el usuario escribe, dispara el setter automáticamente—. Un `rx.foreach` que itera sobre `CourseState.filtered_courses` y llama a `course_card` por cada elemento.

```
rx.input(on_change=CourseState.set_search_query),
rx.foreach(CourseState.filtered_courses, course_card)
```

El resultado: **Sin fetch manual. Sin JSON. Sin Redux. Sin WebSocket.** Todo ese código que en un stack tradicional escribirías tú lo genera y ejecuta Reflex por debajo.

BLOQUE 5 — PÁGINAS, COMPONENTES Y PATRONES DE UI

🕒 1.5 minutos • ~195 palabras • Pág. 6 del PDF • Módulo: DAW

(Pág. 6 del PDF — 30 páginas, 3 patrones de Python)

Las 30 páginas se construyen repitiendo **tres patrones**. Los veis en columnas en pantalla.

Primero, listas reactivas: `rx.cond` comprueba si el estado está cargando —si es así muestra un spinner—; si no, `rx.grid` con `rx.foreach` itera `CourseState.filtered_courses` y llama a `course_card` por cada elemento. Un argumento `columns="3"` hace la rejilla. Cero HTML, cero CSS de grid.

```
rx.cond(CourseState.loading,
  rx.spinner(),
  rx.grid(
    rx.foreach(CourseState.filtered_courses, course_card),
    columns="3"))
```

Segundo, protección de rutas: la función `require_auth` recibe cualquier componente y devuelve `rx.cond(AuthState.is_authenticated, component, login_redirect)`. Una sola línea decide si el usuario ve el componente o acaba en login. Las 33 rutas protegidas usan este mismo patrón.

Tercero, responsive en una línea: `columns=rx.breakpoints(initial="2", sm="3", lg="6")`. Eso es todo. Reflex genera las media queries. Sin CSS, sin media queries escritas a mano, sin JavaScript.

30 páginas • 33 rutas • **0 líneas de CSS manual.**

BLOQUE 6 — SEGURIDAD: AUTENTICACIÓN Y ROLES

🕒 2 minutos • ~260 palabras • Pág. 7 del PDF • Módulo: DAW

(Pág. 7 del PDF — seguridad en 3 decisiones de código)

La seguridad de la plataforma se reduce a **tres decisiones de código**. Las veis en las tres columnas.

Primera decisión —bcrypt con salt (`utils/password.py`): `bcrypt.gensalt()` genera un salt aleatorio único por usuario. `bcrypt.hashpw(password.encode(), salt)` produce un hash irreversible que empieza siempre por `$2b$12$`. Para verificar al hacer login: `bcrypt.checkpw(plain.encode(), stored_hash.encode())`. La misma contraseña da hashes distintos cada vez: sin rainbow tables posibles.

```
salt = bcrypt.gensalt()
hashed = bcrypt.hashpw(password.encode(), salt)

bcrypt.checkpw(plain.encode(), stored_hash.encode())
```

Segunda decisión —error genérico anti-enumeración (`auth_state.py`): si el usuario no existe en la base de datos, el mensaje es `"Email o contraseña incorrectos"`. Si la contraseña falla, el mensaje es exactamente el mismo. El atacante no puede saber si el email existe registrado o no.

```
if not user:
    self.error = "Email o contraseña incorrectos"
    return

if not verify_password(password, user.password):
    self.error = "Email o contraseña incorrectos" # mismo mensaje
    return
```

Tercera decisión —RBAC vía redirección y var computada: tras autenticarse, se comprueba `user.role`. Si es `"admin"`, `yield rx.redirect("/admin/dashboard")`; si es `"instructor"`, su dashboard; si no, el de estudiante. El decorador `@rx.var` expone

`is_user_admin` como propiedad reactiva: la navbar, los menús y los componentes protegidos se actualizan solos cuando el rol cambia.

Tres decisiones. Cero dependencias de seguridad adicionales.

BLOQUE 7 — MERCADO Y OPORTUNIDAD

🕒 1 minuto • ~130 palabras • Pág. 8 del PDF • Módulo: EMPRESA

(Pág. 8 del PDF — mercado e-learning)

El e-learning es uno de los sectores con mayor crecimiento sostenido de la última década: mercado global de 315 mil millones * en 2023, proyección de * **457 mil millones** para 2026 a un CAGR del **13,7%**.

He identificado tres segmentos con potencial real:

Formación técnica: 120.000 profesionales IT en España. Ticket medio 49€–199€. Proyección conservadora Año 1: **13.860€** con 100 estudiantes.

Corporativo B2B: 2,9 millones de PYMEs. Con solo 5 empresas cliente: **12.000€** al año.

Marketplace de instructores: 20 instructores, 50 ventas/mes, comisión 25% → **14.850€** al año.

Los competidores directos —Udemy, Coursera, Domestika, Platzi— dejan desatendido el nicho de **formación técnica accesible y de calidad para el mercado español**. Ese es nuestro hueco.

BLOQUE 8 — MODELO DE NEGOCIO

🕒 1 minuto • ~130 palabras • Pág. 9 del PDF • Módulo: EMPRESA

(Pág. 9 del PDF — modelo de negocio: el motor financiero)

El modelo de negocio tiene tres patas:

1. **Freemium:** cursos básicos gratuitos, premium de 49€ a 99€.
2. **Comisiones:** 25–30% por venta de instructor.
3. **Suscripciones corporativas:** desde 200€/mes.

El **punto de equilibrio son solo 24 ventas al año** —2 cursos al mes—, un umbral altamente accesible.

Un umbral de rentabilidad impulsado por los bajos costes de infraestructura técnica: Reflex Cloud, MongoDB Atlas y el stack Python mantienen los costes operativos en **4.376€ anuales**.

BLOQUE 9 — VIABILIDAD: TÉCNICA, ECONÓMICA Y LEGAL

 1.5 minutos • ~195 palabras • Pág. 10 del PDF • Módulo: FOL + EMPRESA

(Pág. 10 del PDF — matriz de viabilidad 89%)

Forma jurídica: autónomo

Se ha optado por **trabajador autónomo** por tres razones: coste de constitución cero, gestión simplificada y **tarifa plana de 80€/mes** los primeros doce meses —ahorro de 2.676€—. Si el negocio supera los 40.000€ de beneficio, el cambio a SL es más eficiente fiscalmente.

Obligaciones fiscales

- **IVA 21%:** Modelo 303 trimestral + Modelo 390 anual.
- **IRPF:** Modelo 130 trimestral, estimación directa simplificada.
- **RETA:** 80€/mes Año 1; cuota por rendimientos netos a partir del Año 2.

Marco legal

- **RGPD:** datos en MongoDB Atlas con cifrado AES-256 (reposo) y TLS (tránsito), política de privacidad y gestión de cookies.
- **LSSI:** precios visibles, contratación electrónica documentada.
- **Propiedad intelectual:** código bajo **licencia MIT**, dependencias bajo Apache 2.0/PSF.

Resumen económico

Concepto	Valor
Coste operativo anual	4.376€
Punto de equilibrio	24 ventas/año
ROI Año 1 (sin coste desarrollo)	+18% a +506%
Viabilidad global	89% — PROYECTO VIABLE

BLOQUE 10 — AYUDAS AL LANZAMIENTO

 0.5 minutos • ~65 palabras • Pág. 11 del PDF • Módulo: FOL + EMPRESA

(Pág. 11 del PDF — bolsa de ayudas al lanzamiento)

Ayudas reales para el lanzamiento: tarifa plana SS (2.676€), Kit Digital (1.500€), Almería Emprende (3.000€). Total: **7.176€** —prácticamente todos los costes operativos del primer año cubiertos.

El riesgo de lanzamiento queda minimizado desde el primer día.

BLOQUE 11 — CONCLUSIONES Y PRÓXIMOS PASOS

 1 minuto • ~130 palabras • Pág. 12 del PDF • Módulo: FOL + DAW

(Pág. 12 del PDF — conclusiones: elegancia técnica y viabilidad comercial)

Dimensión	Resultado
Técnica	Python full-stack, 0 JS manual
Datos	MongoDB, embedding + referencing
Seguridad	bcrypt, RBAC, RGPD, LSSI
Negocio	Equilibrio en 24 ventas/año
Deploy	Reflex Cloud — en producción
Viabilidad global	89% — ALTAMENTE VIABLE

La aplicación está desplegada y accesible ahora mismo en:

<https://e-learning-jcb-reflex-gray-orca.reflex.run/>

Próximos pasos: Stripe, notificaciones WebSocket, exámenes/certificados, analítica con IA.

Python es suficiente para construir una plataforma web profesional, segura y comercialmente viable.

A continuación, la demostración en vivo.

APÉNDICE — RESPUESTAS A PREGUNTAS FRECUENTES

¿Por qué autónomo y no Sociedad Limitada?

Para el primer año, el autónomo es más ágil y barato. La tarifa plana reduce la SS a 80€/mes. Si los beneficios superan 40.000€, el cambio a SL es más eficiente fiscalmente.

¿Qué pasa con el RGPD y los datos de los estudiantes?

Los datos se almacenan en MongoDB Atlas con cifrado AES-256 en reposo y TLS en tránsito. La plataforma incluye política de privacidad, gestión de cookies y formularios con consentimiento explícito.

¿No es más lento que usar React directamente?

En producción, Reflex compila a React optimizado. La diferencia de rendimiento es mínima para este tipo de aplicación. La ganancia en velocidad de desarrollo es enorme.

¿Se puede usar SQL en lugar de MongoDB?

Sí. Reflex funciona con cualquier base de datos. MongoDB se eligió por la flexibilidad de esquema para cursos con estructura variable.

¿Cómo se hace el deploy?

Reflex Cloud. La app está desplegada en: <https://e-learning-jcb-reflex-gray-orca.reflex.run/>

¿Qué fue lo más difícil de desarrollar?

El sistema de estados con herencia y el bug del doble `on_mount`. Entender cómo el estado reactivo se sincroniza con el frontend requiere cambiar la forma de pensar respecto al desarrollo web tradicional.

Guion oral — Exposición teórica 15 minutos — E-Learning JCB

~1.950 palabras • ~130 palabras/minuto • Módulos: DAW • FOL • EMPRESA